Legacy System Takeover Assessment — Banking Management System

Oleksandr Serhiichyk · Senior .NET Engineer home assignment · 26 July 2026

Submission: `github.com/o-serhiichyk/banking-system-desktop` · upstream under assessment: `github.com/ali7haider/banking-system-desktop`

This is a WinForms application whose business rules live in click handlers, whose session is a static mutable object, and whose persistence layer is seven comma-delimited text files. Its dominant risks are authorization by username string, three balance models that disagree on the normal path, a ledger recording nothing about who changed what, and one keystroke that can stop the application starting for everyone. I added one deliberately scoped increment — an append-only audit trail — and validated the AI-assisted analysis behind it by execution rather than agreement.

Task 1 · Reverse engineering the system

Architecture

Three folders imply a layered design; the running program does not have one. The principal path is **WinForms event handler → static BL/DL state → comma-delimited file**; the layers are conventions, not boundaries.

UI — 33 forms and user controls: `Form1` (login), `CusForm`, `AdminWindow`, five search controls

▼ Click

Event handlers — where the business rules actually are: parse input, compute the balance, decide, then write files (`TransactMoneyCus.cs:38-89`)

▼ static calls, bypassing BL/

DL/ — `AdminDL`, `CustomerDL`, `MUserDL`: every member `static`, over static mutable session state and the customer list (`DL/AdminDL.cs:13-17`)

▼ bare relative paths

Storage — seven comma-delimited `.txt` files, plus `auditTrail.csv` added by this work

`LogIn`'s constructor loads the whole datastore before `Application.Run`; a bad row kills the process before any window exists (`Form1.cs:18-26`)

① **Rules in the UI:** no service layer, no seam — every money rule is inside a `Click` handler, so none of it is unit-testable

② **Layers are not boundaries:** handlers call DL directly and open files themselves, and `BL/` (`Admin`, `Customer`, `MUser`) decides nothing. ③ **Globally mutable state:** one session per process, never cleared on logout

④ **No transaction, schema, escaping or locking.** Filenames are relative literals at 16 sites, so the working directory picks the dataset

Main components and key business flows

Two consoles sit behind one login — a customer window (deposit, withdraw, transfer, balance, history) and an admin window (customer CRUD, searches, bank totals) — both driven by the flows below.

UI entry	State change	Files written	Principal risk
Create customer Add User → Confirm	New `Admin` and `MUser` appended to two static lists (`AddUser.cs:90,92`)	`customers.txt`, `Users.txt` — two independent appends	Fields are concatenated unescaped (`DL/AdminDL.cs:96`): one comma shifts the record and the next launch fails
Deposit Deposit Money → Confirm	`Current.TotalMoney += amount` in memory only (`DepositMoneyCus.cs:61`)	`depositHistory.txt` — append	The balance reaches `customers.txt` only at logout; the date is taken from a picker's text, not the clock (`:59`)
Withdraw Withdraw Money → Confirm	`Current.TotalMoney += amount` — **adds** (`WithDrawMoneyCus.cs:30`)	`withDrawHistory.txt` — append	A withdrawal credits the account, and nothing checks the amount or the available funds
Transfer Transact Money → Confirm	Recipient matched by == on a `double`; **neither balance changes** (`TransactMoneyCus.cs:36-90`)	`transactHistory.txt` then `sendMoneyPath.txt` — uncoordinated	Money never moves; a failure between the two writes leaves a half-recorded transfer with no rollback

Data storage

There is no database. Seven comma-delimited text files hold customers, credentials and four history ledgers; the trail adds an eighth. Every path is a bare relative literal at 16 sites, so the working directory decides which dataset is used, and the repository ships two divergent copies. Records are positional and read by field index: no schema, no escaping, no type enforcement, no transactional boundary, no locking, and nothing in the application that backs a file up or restores one. The store is read into static lists at login and rewritten from memory at logout (`DL/AdminDL.cs:100-109`).

Main technical risks

The findings resolve into five categories; the ranked five are in Task 3 and are not repeated here.

Authorization — a compiled-in credential pair and a username-string admin check, over passwords held in cleartext.

Ledger integrity — three disagreeing balance definitions, a withdrawal that credits, and transfers that move nothing.

Traceability — no record of who acted, and the dates that exist are operator-chosen and therefore falsifiable.

Datastore fragility — unescaped delimiters, non-atomic multi-file writes, no locking, no application-level backup or recovery path, and a store that lives in a build-output directory.

Change safety — no test project and no seam to add one, because every money rule sits inside a `Click` handler.

Areas requiring further investigation

Deployment and reproducible builds. The solution builds incrementally, but a **clean** build fails on this toolchain because committed generated resources are load-bearing (`MSB3823`/`MSB3822`); a successful incremental build must not be read as build health. The mechanism and the workaround are in the README.

The datastore. Who owns it in production, what backup and recovery exist, whether two instances are ever run concurrently, and which of the three balances is meant to be authoritative.

Identity. What roles were intended beyond customer and administrator, how existing credentials would be migrated to hashes, and who has file-system access to the directory the data sits in.

AI tools, workflow and validation

I used Claude Code with Opus for repository analysis and implementation assistance. Claude Code with Fable and OpenAI Codex with GPT-5.6 Sol provided independent adversarial reviews. I adjudicated their challenges through source inspection and runtime validation.

The workflow was to form hypotheses from the code and commit them before running any model, conduct a broad analysis pass, require every claim to cite a file and line, and then independently verify consequential findings through source inspection or execution. I tested behavioural claims by running the application against a snapshot of the datastore and comparing the resulting files. This process changed conclusions rather than merely confirming them. For example, runtime testing showed that a suspected balance-recalculation defect occurred once per login, not on every visit to the balance screen. It also turned one of my own inferences into demonstrated evidence: I confirmed that a malformed record could prevent the application from starting by launching the same executable against datasets differing by only that record.

Model agreement was never evidence; where two models converged on a wrong reading, only execution separated them. Feature selection, the risk ranking, severity calls and every adjudication remained my decisions — one model-proposed improvement was built, measured, then deliberately reverted as out of scope.

Task 2 · Missing and incomplete functionality

Complexity reflects implementation breadth, migration risk and validation effort: Simple is localized, Medium is bounded but cross-cutting, and Complex changes core data, security or operational architecture.

Capability	What is missing	Business value	Complexity
Pre-transaction validation guard	No funds, amount or self-transfer check on any money path (`TransactMoneyCus.cs:41-53`)	Every accepted transaction is one the bank can stand behind, instead of one it has to unwind	Medium
Account status and non-destructive closure	Only a hard delete exists; history is orphaned and the account number is reissued	A relationship can end while its history is retained, and an account can be frozen rather than destroyed	Complex
Credential protection at rest and on screen	Passwords stored, displayed, searched and matched in cleartext (`DL/MUserDL.cs:84`)	Customer credentials stay secret from staff and from anyone with file access — the baseline expectation of a bank	Medium
Explicit role attribute	Authorization is one username comparison; no record carries a role (`DL/MUserDL.cs:42-49`)	Operator rights are granted deliberately and can be revoked, rather than implied by a name	Medium
Append-only audit trail ← selected	Nothing records who changed a balance, when, or from what value	Every balance change carries an actor, a time and a before/after value, so disputes can be answered	Medium
Atomic, recoverable writes with backup	Rewrites truncate and re-emit from memory; no temp file, no copy, no lock	Customer records survive a crash or a bad write, with a known recovery path	Medium
Transaction receipt and account statement	No reference number, no combined chronological statement, no export	Customers get proof of a transaction and a statement they can reconcile; staff can produce one on request	Medium
Product differentiation by account type	Account type is captured, stored and displayed, and branches no behaviour	The bank can offer and price distinct products — interest, fees, minimum balances	Complex
Transaction and balance limits	No ceiling, daily cap, minimum balance or velocity check exists	Exposure per transaction and per day is bounded by policy, containing error and fraud before either is noticed	Medium
Multiple accounts per customer	Customer and account are the same record, keyed by one account number	One customer can hold several accounts — the ordinary retail relationship — without duplicate identities	Complex

Why the audit trail, out of the Mediums

Observability before correction. In a legacy takeover you cannot safely repair what you cannot see, and nothing here records who did anything. Every other Medium changes what the application does; this one only observes, so its worst failure mode is a wrong log file rather than a refused legitimate withdrawal. It is genuinely Medium, has a small blast radius (a new writer plus call sites, no existing statement altered), presents a pure writer surface that is unit-testable without a UI, and produces the evidence needed to correct the higher-ranked risks safely. Its reach is the widest on the slate: with before/after values and an actor on every recorded operation, roughly ten findings across five failure modes become detectable rather than silent.

Scoped deliberately as the first useful increment: a local, append-only diagnostic record written beside the datastore. The production form of this capability — centralized, durable, access-controlled and tamper-evident — is a Complex programme, and holding the first increment to the diagnostic record is what keeps it Medium and safe to add to a system with no tests.

Task 3 · Code review — the five most important risks

I ranked by likelihood, magnitude of business harm and recoverability, weighting demonstrated failures above speculative maintainability concerns. **Runtime** means observed by executing the application and diffing the datastore; **Source**, established by reading the code.

One placement note before the table. Rank 4 sits below three silent failures deliberately: its blast radius is total, but the failure is loud, immediately obvious and recoverable by someone who knows the defect exists, where ranks 1–3 harm the bank without anyone noticing. Money and identity held in `double` sits just below the line on the same test — certain, but a precondition for ever **proving** rank 2 correct rather than a failure observed today; it is catalogued in `docs/FINDINGS.md`.

#	Location	Risk	Business impact	Recommended fix	Evidence
1	`DL/MUserDL.cs :26-31, :42-49` `Form1.cs:65-78`	A hardcoded `Admin/1234` pair is accepted before the user store is read, and `isAdmin` returns true for anyone named `admin` — no password, no role	Any customer can hold full operator rights — every customer's PII and password, arbitrary balance edits, account deletion — obtained with no secret and leaving no trace. The compiled-in credential cannot be revoked without a code change	Persist a role and have `isAdmin` read it; move the admin account into the user store, hashed; delete the literal	**Runtime**
2	`DL/CustomerDL.cs :198-248, :253-259` `WithDrawMoneyCus.cs :30` `TransactMoneyCus.cs :36-90`	Three balance definitions run at once and all three are wrong: withdrawal adds, the recompute mutates state and re-applies history per login, the display omits the opening deposit, transfers settle nothing	The bank cannot state what a customer holds: one account read 2000 on screen, 13000 in memory and 9000 on disk at the same instant, and a stored balance drifted by 2000 with nobody touching it. Transfers are recorded but settle nothing	One authoritative calculation; make the `calculate*` functions pure; settle transfers at commit	**Runtime**
3	`DepositMoneyCus.cs :59` (+2 handlers) `EditCustomer.cs :53-74` `ViewCustomer.cs :105-125`	Money records take their date from a form control rather than the clock, so entries are backdatable; privileged edits and deletes record no operator, previous value or time	After a discrepancy, the first question — who changed this, when, from what — has no answer, and an escalated operator is indistinguishable from a legitimate one	System-generated ledger timestamps; an append-only trail carrying operator, before/after values and time	**Source**
4	`DL/AdminDL.cs :96,105` (+7 sites) `:73` `Form1.cs:18-26`	Records are concatenated unescaped, so one comma in a name shifts every field; `read_data` then parses the shifted field at launch, in a constructor that runs before `Application.Run`	One ordinary name entry takes the whole bank offline: **no customer and no administrator can start the application**, and with no application-level backup and no admin tooling, recovery means hand-editing the data file	Escape on write; reject the delimiter on input; guard the launch-time parse so a bad row quarantines itself instead of killing the process	**Runtime**
5	`BMS WinForm.csproj` `DL/CustomerDL.cs :253-259`	No test project and no seam to add one: every money rule sits in a `Click` handler or a `static` method over global state, opening files by relative path	Not future breakage — present defects. Rank 2 rests on two one-line arithmetic errors a single unit test would catch, and every fix to ranks 1–4 ships unverified	A test project plus `InternalsVisibleTo` (enacted in Task 4), and extraction of the money rules out of the handlers	**Source**

Task 4 · Implementation — basic append-only audit trail

Four pieces. Session state captures the operator the system believes is acting, set after the authentication check and before the admin branch so both paths populate it (`Form1.cs:72`). A fixed twelve-column record defines the schema — time, operation id, operator, event, subject and counterparty accounts, amount, balance before and after, target file and changed fields — with passwords redacted to the **fact** of a change. An append-only writer applies RFC 4180 quoting, writes the header once, renders numbers round-trip exactly and culture-invariantly, and swallows every exception. Seven UI capture sites record deposit, withdrawal,

transfer, customer create, edit and delete, and the logout balance write; a transfer produces a correlated **pair** sharing an operation id, because its two file writes are independent and one can succeed alone.

Source and build. New code: `DL/AuditWriter.cs`, `DL/AuditRecord.cs`, `DL/AuditSession.cs`, the seven capture sites, and tests in `tests/BMS.Tests/`. Build with `dotnet build "BMS WinForm.sln"` and run the executable **from** `BMS WinForm/bin/Debug/`, since every data path is relative and the working directory selects the dataset; `dotnet test` runs the suite. The README carries these commands, the pre-existing clean-build constraint and the data-restore step; `docs/specs/AUDIT_TRAIL.md` is the full contract.

Assumptions, scope and known limitations

Assumptions. The login's username identifies the **system-believed** actor, not a trustworthy authenticated principal — rank 1 means it may have been obtained by escalation. The process can append a file beside the datastore. The machine clock is useful for diagnostic timestamps, and is neither authoritative nor tamper-resistant.

Scope and design decisions. Detection and investigation support — not prevention, and not ledger correction. Capture happens after the business operation, so fail-closed semantics are unavailable. An audit-write failure must never block or alter a completed operation: the writer swallows exceptions, since a throwing append here would turn one deposit into two.

Known limitations. Coverage is seven call sites, not a chokepoint — the absence of an entry is not evidence that an operation did not occur. Recording is best-effort: failed appends are dropped silently, and concurrent instances lose a measured fraction. The local CSV sits beside the data with the same permissions: not tamper-evident, not an authoritative ledger.

Validation

Automated. NUnit tests cover the writer's pure surface only — quoting and escaping, exact and culture-invariant number rendering, column order, header-once, password redaction, and the swallow-on-failure contract. They do not reach the capture sites at all: deleting any single `AuditWriter.Append` call leaves the whole suite green.

Manual. The capture sites are validated by execution instead, because every one is a `Click` handler with no test seam — rank 5 constraining the work rather than being designed around. **The quick check, about 20 seconds:** delete `auditTrail.csv` from `BMS WinForm/bin/Debug/`, start the application from that directory, log in as `Haider` / `15`, deposit any amount, log out. The file should then hold a header and exactly **two** data rows — `Deposit` then `LogoutBalanceWrite` — both with `operator` = `Haider` and before/after balances. The full seven-site scenario (README, and `docs/specs/AUDIT_TRAIL.md` §7.1) produces eight data rows, with the transfer pair sharing an operation id; re-running it is the only check in the repository that would notice a capture site that stopped firing.

Ownership. The defects above are the original application's; this change adds only the per-call-site coverage limit, the best-effort append, and a file that is not tamper-evident. Validating it also produced new evidence about the original system: a credential that survives customer deletion, and the startup failure behind rank 4.

Supporting detail, all in the submitted repository: `docs/FINDINGS.md` (51 findings and the ranking), `docs/CAPABILITIES.md` (the slate of ten and the rejected candidates), `docs/specs/AUDIT_TRAIL.md` (implementation contract and limits), `docs/ANALYSIS_LOG.md` (append-only record of AI passes, probes and attribution), and the README (build, test and validation commands).